Omar Ahmed

Packet Sniffing and Spoofing

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/Sniffing_Spoofing/Sniffing_Spoofing.pdf]

Set 1:

Task 1.1A:

Finding network interface name:

```
root@VM:/volumes# ifconfig
br-cb66066b4635: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::42:d4ff:fe21:ae46  prefixlen 64  scopeid 0x20<link>
        ether 02:42:d4:21:ae:46  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 389  bytes 44970 (44.9 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- The new iface will be updated in sniffer.py: br-cb66066b4635

```
root@VM:/volumes# chmod a+x sniffer.py
root@VM:/volumes# ./sniffer.py
^Croot@VM:/volumes# su seed
seed@VM:/volumes$ ./sniffer.py
Traceback (most recent call last):
  File "./sniffer.py", line 6, in <module>
    pkt = sniff(iface='br-cb66066b4635', filter='icmp', prn=print_pkt)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 1036, in sniff
    sniffer._run(*args, **kwargs)
  File "/usr/local/lib/python3.8/dist-packages/scapy/sendrecv.py", line 906, in _run
    sniff_sockets[L2socket(type=ETH_P_ALL, iface=iface,
  File "/usr/local/lib/python3.8/dist-packages/scapy/arch/linux.py", line 398, in __init__
    self.ins = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(type))  # noqa: E5
  File "/usr/lib/python3.8/socket.py", line 231, in __init__
    _socket.socket.__init__(self, family, type, proto, fileno)
PermissionError: [Errno 1] Operation not permitted
seed@VM:/volumes$ su root
Password:
root@VM:/volumes#
```

- The sniffer.py program does not return any errors when running in root, but in the seed account without root privilege, we get a Permission Error

Task 1.1B:

Capture only ICMP Packet:

```
pkt = sniff(iface='br-cb66066b4635', filter='icmp', prn=print_pkt)
```

- The example code already filters for ICMP packets only.

Capture any TCP packet that comes from a particular IP and with a destination port number 23.

```
pkt = sniff(iface='br-cb66066b4635', filter='tcp and src host 10.9.0.1 and dst port 23', prn=print_p
```

Capture packets comes from or to go to a particular subnet. You can pick any subnet, such as 128.230.0.0/16; you should not pick the subnet that your VM is attached to.

```
pkt = sniff(iface='br-cb66066b4635', filter='net 128.230.0.0/16', prn=print_pk
```

- Subnet is specified using *net*

Task 1.2:

```
[11/14/24]seed@VM:~/.../Labsetup$ dockps
6e89b51c1535  hostA-10.9.0.5
004dacb06eec  seed-attacker
e1aa4fbbdd38  hostB-10.9.0.6
b00e913285f3  www-10.9.0.80
```

volumes > icmpPacket.py > ...

```
#!/usr/bin/env python3
from scapy.all import*

a = IP()
a.dst = '10.9.0.5'
b=ICMP()
p=a/b
send(p)
```

- a.dst is changed to 10.9.0.5 (hostA's IP)

```
root@VM:/volumes# sniffer.py
###[ Ethernet ]###
  dst       = 02:42:0a:09:00:05
  src       = 02:42:d4:21:ae:46
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 28
     id        = 1
     flags     =
     frag      = 0
     ttl       = 64
     proto     = icmp
     chksum    = 0x66c9
     src       = 10.9.0.1
     dst       = 10.9.0.5
     \options   \
###[ ICMP ]###
        type      = echo-request
        code      = 0
        chksum    = 0xf7ff
        id        = 0x0
        seq       = 0x0
```

```
root@VM:/volumes# icmpPacket.py
.
Sent 1 packets.
```

- sniffer.py picks up the sent ICMP packet.

15	2024-11-14 14:2…	10.9.0.1	10.9.0.5	ICMP	42	Echo (ping) request id=0x0000, seq=0/0,
16	2024-11-14 14:2…	10.9.0.5	10.9.0.1	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0,
17	2024-11-14 14:2…	02:42:0a:09:00:05	02:42:d4:21:ae:46	ARP	42	Who has 10.9.0.1? Tell 10.9.0.5
18	2024-11-14 14:2…	02:42:d4:21:ae:46	02:42:0a:09:00:05	ARP	42	10.9.0.1 is at 02:42:d4:21:ae:46

- Wireshark shows that the request is accepted because an echo reply is sent to the spoofed IP address, 10.9.0.1
    - There are ARP requests to tell IP 10.9.0.5‘who’ has 10.9.0.1

Task 1.3:

volumes > traceroute.py > ...

```python
#!/usr/bin/env python3
from scapy.all import *

def traceroute(dst, max_ttl):
    for ttl in range(1, max_ttl + 1):
        pkt = IP(dst=dst, ttl=ttl) / ICMP()
        reply = sr1(pkt, verbose=0, timeout=2)

        if reply is None:
            print(f"{ttl}: No reply")
        elif reply.type == 0:
            print(f"{ttl}: {reply[IP].src} (Destination reached)
            break
        else:
            print(f"{ttl}: {reply[IP].src}")

if __name__ == "__main__":
    traceroute("1.2.3.4", 60)
```

- traceroute.py automates the process described in the lab description, taking in the destination ip address, and the max number of ttl to loop up to.

```
root@VM:/volumes# traceroute.py
1: 10.0.2.2
2: 192.168.2.1
3: 142.124.42.101
4: 142.124.42.101
5: 142.124.42.101
6: 142.124.42.101
7: 142.124.42.101
8: 142.124.42.101
9: 142.124.42.101
10: 142.124.42.101
11: 142.124.42.101
12: 142.124.42.101
13: 142.124.42.101
14: 142.124.42.101
15: No reply
16: No reply
17: No reply
18: No reply
19: No reply
```

Task 1.4:

volumes > sniffandspoof.py > ...

```python
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
        if (pkt[2].type == 8): #ICMP packet


                a = IP()
                a.src = pkt[1].dst #set the source of the packet to the destination of the sniffed packet
                a.dst = pkt[1].src #set the destination of the spoof to be the source of the sniffed packet (we are ech
                b=ICMP(type=0, id=pkt[2].id, seq=pkt[2].seq) #type=0 is an echo reply
                load = pkt[3].load
                p=a / b / load #the payload and sequence id need to be extracted as well
                send(p)

pkt = sniff(iface='br-cb66066b4635', filter='icmp', prn=print_pkt)
```

- sniffandspoof.py combines tasks 1.1 and 1.2 by sniffing and checking if an ICMP packet comes through, and if one does, then a spoofed echo reply is sent by using the data from the sniffed packet.

volumes > sniffandspoof.py > ...

```python
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
        if ICMP in pkt and (pkt[ICMP].type == 8): #ICMP packet


                a = IP()
                a.src = pkt[IP].dst #set the source of the packet to the destination of the sniffed packet
                a.dst = pkt[IP].src #set the destination of the spoof to be the source of the sniffed packet (we are ec
                b=ICMP(type=0, id=pkt[2].id, seq=pkt[2].seq) #type=0 is an echo reply
                load = pkt[Raw].load
                p=a / b / load #the payload and sequence id need to be extracted as well
                #print source IP and destination IP of our spoofed packet
                print("Source: ", p[IP].src)
                print("Destination:", p[IP].dst)
                send(p)

pkt = sniff(iface=['enp0s3','lo', 'br-cb66066b4635'], filter='icmp', prn=print_pkt)
```

PROBLEMS 14 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
rtt min/avg/max/mdev = 4.196/17.065/57.880/14.216 ms
root@VM:/volumes# ping 1.2.3.4
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
From 142.124.42.101 icmp_seq=1 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=26.0 ms
From 142.124.42.101 icmp_seq=2 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=2 ttl=64 time=35.9 ms
From 142.124.42.101 icmp_seq=3 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=3 ttl=64 time=34.2 ms
From 142.124.42.101 icmp_seq=4 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=4 ttl=64 time=24.3 ms
From 142.124.42.101 icmp_seq=5 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=5 ttl=64 time=32.1 ms
From 142.124.42.101 icmp_seq=6 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=6 ttl=64 time=37.7 ms
From 142.124.42.101 icmp_seq=7 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=7 ttl=64 time=27.8 ms
From 142.124.42.101 icmp_seq=8 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=8 ttl=64 time=33.6 ms
From 142.124.42.101 icmp_seq=9 Destination Net Unreachable
64 bytes from 1.2.3.4: icmp_seq=9 ttl=64 time=18.8 ms
```

```
Sent 1 packets.
Source:  1.2.3.4
Destination: 10.0.2.15
.
Sent 1 packets.
Source:  1.2.3.4
Destination: 10.0.2.15
.
Sent 1 packets.
Source:  1.2.3.4
Destination: 10.0.2.15
.
Sent 1 packets.
Source:  1.2.3.4
Destination: 10.0.2.15
.
Sent 1 packets.
Source:  1.2.3.4
Destination: 10.0.2.15
.
Sent 1 packets.
```

- Pinging 1.2.3.4, sniffandspoof.py successfully sends echo reply packets. This is expected behavior because 1.2.3.4 is a non existing host on the internet, we are simply just spoofing reply packets.

```
PROBLEMS 14   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

From 10.9.0.1 icmp_seq=8 Destination Host Unreachable
From 10.9.0.1 icmp_seq=9 Destination Host Unreachable
From 10.9.0.1 icmp_seq=10 Destination Host Unreachable
From 10.9.0.1 icmp_seq=11 Destination Host Unreachable
From 10.9.0.1 icmp_seq=12 Destination Host Unreachable
From 10.9.0.1 icmp_seq=13 Destination Host Unreachable
From 10.9.0.1 icmp_seq=14 Destination Host Unreachable
From 10.9.0.1 icmp_seq=15 Destination Host Unreachable
From 10.9.0.1 icmp_seq=19 Destination Host Unreachable
From 10.9.0.1 icmp_seq=20 Destination Host Unreachable
From 10.9.0.1 icmp_seq=21 Destination Host Unreachable
From 10.9.0.1 icmp_seq=22 Destination Host Unreachable
From 10.9.0.1 icmp_seq=23 Destination Host Unreachable
From 10.9.0.1 icmp_seq=24 Destination Host Unreachable
From 10.9.0.1 icmp_seq=28 Destination Host Unreachable
From 10.9.0.1 icmp_seq=31 Destination Host Unreachable
From 10.9.0.1 icmp_seq=32 Destination Host Unreachable
From 10.9.0.1 icmp_seq=33 Destination Host Unreachable
From 10.9.0.1 icmp_seq=34 Destination Host Unreachable
From 10.9.0.1 icmp_seq=35 Destination Host Unreachable
From 10.9.0.1 icmp_seq=36 Destination Host Unreachable
```

- Pinging 10.9.0.99, a non-existing host on the LAN
    - We get Destination Host Unreachable

```
root@VM:/volumes# ip route get 10.9.0.1
local 10.9.0.1 dev lo src 10.9.0.1 uid 0
    cache <local>
root@VM:/volumes# ip route get 1.1.1.1
1.1.1.1 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 0
    cache
```

- This is because there is no route to 10.9.0.99

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=4.95 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=22.4 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=4.11 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=25.9 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=3 ttl=114 time=4.14 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=26.3 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=4 ttl=114 time=4.50 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=27.8 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=5 ttl=114 time=3.77 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=64 time=34.8 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=6 ttl=114 time=3.63 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=64 time=36.1 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=7 ttl=114 time=3.73 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=64 time=27.1 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=8 ttl=114 time=3.74 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=64 time=40.2 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=9 ttl=114 time=4.04 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=64 time=35.4 ms (DUP!)
64 bytes from 8.8.8.8: icmp_seq=10 ttl=114 time=3.92 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=64 time=41.7 ms (DUP!)
```

```
Sent 1 packe
Source:  8.8
Destination:
.
Sent 1 packe
Source:  8.8
Destination:
.
Sent 1 packe
Source:  8.8
Destination:
.
Sent 1 packe
Source:  8.8
Destination:
.
Sent 1 packe
Source:  8.8
Destination:
.
Sent 1 packe
```

- Pinging 8.8.8.8, an existing host on the internet
    - We see that there are (DUP!), this is because both my sniffandspoof.py program and the actual host are sending echo replies.

Set 2:

Task 2.1A

```
root@VM:/volumes# ./sniffer
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.6
Got a packet
Source IP: 10.9.0.6
Destination IP: 10.9.0.1
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.6
Got a packet
Source IP: 10.9.0.6
Destination IP: 10.9.0.1
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.6
Got a packet
Source IP: 10.9.0.6
Destination IP: 10.9.0.1
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.6
Got a packet
Source IP: 10.9.0.6
```

We ran the sniffer program in the container and pinged the address 10.9.0.6

```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <netinet/ip.h>

void got_packet(u_char *args, const struct pcap_pkthdr *hea
der, const u_char *packet){
        printf("Got a packet\n");

        // Assuming IPv4 packets, skip the ethernet header
(first 14 bytes)
        const u_char *ip_header = packet + 14;

        // Extract source and destination IP addresses (off
set of 12 and 16 in the IP header)
        struct in_addr src_ip, dest_ip;
        src_ip.s_addr = *(uint32_t *)(ip_header + 12);
        dest_ip.s_addr = *(uint32_t *)(ip_header + 16);
        printf("Source IP: %s\n", inet_ntoa(src_ip));
        printf("Destination IP: %s\n",inet_ntoa(dest_ip));
}

int main(){
        pcap_t *handle;
        char errbuf[PCAP_ERRBUF_SIZE];
        struct bpf_program fp;
        char filter_exp[] = "ip"; // Filter to capture only
 IP packets
                                                5,22          Top
```

```
        char filter_exp[] = "ip"; // Filter to capture only
 IP packets
        bpf_u_int32 net;

        handle = pcap_open_live("br-acdb473042fd", BUFSIZ,
l, 1000, errbuf);
        if(handle == NULL){
                fprintf(stderr, "Could not open: %s\n", err
buf);
                return 1;
        }

        if(pcap_compile(handle, &fp, filter_exp, 0, net) ==
 -1){
                fprintf(stderr, "Error compiling filter: %s
\n", pcap_geterr(handle));
                return 1;
        }

        if(pcap_setfilter(handle, &fp) == -1){
                fprintf(stderr, "Error setting filter: %s\n
', pcap_geterr(handle));
                return 1;
        }

        // Start packet capture
        pcap_loop(handle, -1, got_packet, NULL);

        pcap_close(handle);
                                        29,15-22      88%
```

- **Initialize the Packet Capture Interface**:

- Use pcap_open_live() to specify the network interface, snapshot length, promiscuous mode, and timeout for packet capture. It returns a pcap_t handle for subsequent operations.

Compile a Filter (Optional):

- If only specific packets are to be captured, use pcap_compile() to compile a filter expression into BPF (Berkeley Packet Filter) bytecode.

Set the Filter:

- Apply the compiled filter to the capture session using pcap_setfilter().

Start Packet Capture Loop:

- Use pcap_loop() to continuously capture packets. For each packet, a callback function (e.g., got_packet()) is invoked to process the packet data.

Process Captured Packets:

- The callback function extracts and processes relevant packet information, such as headers or payloads.

Close the Capture Session:

- Use pcap_close() to release resources associated with the capture session.

2.

Why Root Privileges Are Required:

- Packet sniffing requires direct access to the network interface to capture raw packets. This operation bypasses the operating system's networking stack, which is restricted to privileged users to ensure security and prevent misuse (e.g., unauthorized monitoring of network traffic).

Where the Program Fails:

- Without root privileges, pcap_open_live() fails to open the network interface for live capture. It typically returns a NULL handle, and an error message is stored in the errbuf buffer.

3.

Turning On and Off Promiscuous Mode:

- **Promiscuous Mode Enabled (pcap_open_live(..., 1, ...))**:
    - The network interface captures all packets on the network, regardless of whether they are addressed to the host machine or not.
- **Promiscuous Mode Disabled (pcap_open_live(..., 0, ...))**:
    - The network interface only captures packets explicitly addressed to the host machine.
- When promiscuous mode was disabled, the program didn’t show any packets

Task 2.1B

Updated code:

```
        // Assuming IPv4 packets, skip the ethernet head
er (first 14 bytes)
        const u_char *ip_header = packet + 14;

        int protocol = *(ip_header + 9);

        // Extract source and destination IP addresses (
offset of 12 and 16 in the IP header)
        struct in_addr src_ip, dest_ip;
        src_ip.s_addr = *(uint32_t *)(ip_header + 12);
        dest_ip.s_addr = *(uint32_t *)(ip_header + 16);
        printf("Source IP: %s\n", inet_ntoa(src_ip));
        printf("Destination IP: %s\n",inet_ntoa(dest_ip)
);
        if(protocol == 1){
                printf("Protocol: ICMP\n");
        } else if (protocol == 6){
                printf("Protocol: TCP\n");

        const u_char *tcp_header = ip_header + ((*ip_hea
der & 0x0F)* 4);
        int dst_port = ntohs(*(uint16_t *)(tcp_header +
2));
        printf("Destination Port Number: %d\n", dst_port
);} else {
                printf("Protocol: Other (Not ICMP or TCP
)\n");
        }
        printf("\n");
```

The code checks the **protocol field** (at offset 9 in the IP header) and categorizes packets based on the following:

- **Protocol = 1**: Identified as ICMP.
- **Protocol = 6**: Identified as TCP.

For this example we used the command ping -c 1 10.9.0.6 and this was the output:

```
root@VM:/volumes# ./sniffer
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.6
Protocol: ICMP

Got a packet
Source IP: 10.9.0.6
Destination IP: 10.9.0.1
Protocol: ICMP
```

To capture TCP packets we used the filter “tcp and (dst portrange 10-100)”. To test the filter we used the command telnet 10.9.0.6 15. Here is the output:

```
root@VM:/volumes# ./sniffer
Got a packet
Source IP: 10.9.0.1
Destination IP: 10.9.0.5
Protocol: TCP
Destination Port Number: 15
```

We modified the filter expression to see the output of

1. ICMP packets between 2 hosts

2. TCP packets with destination ports between 10-100

Task 2.1C

```
/* Ethernet header */
struct ethheader {
    u_short ether_type; /* Protocol type (e.g., IPv4: 0x0800)
*/
};

/* IP Header */
struct ipheader {
    unsigned char      iph_ihl:4, iph_ver:4; // IP header leng
th and version
    unsigned char      iph_tos;              // Type of service
    unsigned short int iph_len;              // Packet length
    unsigned char      iph_protocol;         // Protocol type (
TCP, UDP, etc.)
    struct  in_addr    iph_sourceip;         // Source IP
    struct  in_addr    iph_destip;           // Destination IP
};

/* TCP Header */
struct tcpheader {
    unsigned short th_sport; /* Source port */
    unsigned short th_dport; /* Destination port */
    unsigned char th_offx2;  /* Data offset */
};

                                              23,18          8%
```

```
void print_payload(const u_char *payload, int len) {
    printf("Payload: ");
    for (int i = 0; i < len; i++) {
        if (isprint(payload[i])) {
            printf("%c", payload[i]);
        } else {
            printf(".");
        }
    }
    printf("\n");
}

void got_packet(u_char *args, const struct pcap_pkthdr *header
, const u_char *packet) {
    const struct ethheader *eth = (struct ethheader *)packet;

    // Check if packet is IPv4
    if (ntohs(eth->ether_type) == 0x0800) {
        const struct ipheader *ip = (struct ipheader *)(packet
 + sizeof(struct ethheader));
        int ip_header_length = ip->iph_ihl * 4;

        // Check if protocol is TCP
        if (ip->iph_protocol == IPPROTO_TCP) {
@@@
                                                34,16         39%
```

```
            const struct tcpheader *tcp = (struct tcpheader *)
(packet + sizeof(struct ethheader) + ip_header_length);
            int tcp_header_length = (tcp->th_offx2 >> 4) * 4;

            const char *payload = (char *)(packet + sizeof(str
uct ethheader) + ip_header_length + tcp_header_length);
            int payload_length = ntohs(ip->iph_len) - (ip_head
er_length + tcp_header_length);

            if (payload_length > 0) {
                printf("Source: %s Port: %d\n", inet_ntoa(ip->
iph_sourceip), ntohs(tcp->th_sport));
                printf("Destination: %s Port: %d\n", inet_ntoa
(ip->iph_destip), ntohs(tcp->th_dport));
                print_payload((const u_char *)payload, payload
_length);
            }
        }
    }
}

int main() {
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
Applications                                        64,1          67%
```

```
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "tcp port 23"; // Capture only telnet
traffic
    bpf_u_int32 net;

    // Open live pcap session on NIC (update interface name as
 needed)
    handle = pcap_open_live("br-acdb473042fd", BUFSIZ, 1, 1000
, errbuf);
    if (handle == NULL) {
        fprintf(stderr, "Could not open device: %s\n", errbuf)
;
        return 1;
    }

    // Compile and apply the filter
    if (pcap_compile(handle, &fp, filter_exp, 0, net) == -1 ||
 pcap_setfilter(handle, &fp) == -1) {
        fprintf(stderr, "Could not apply filter: %s\n", pcap_g
eterr(handle));
        return 1;
    }

                                                74,44         89%
```

When running telnet 10.9.0.6, it printed the raw data of the login attempt. The password was 1234 in this case:

```
Source: 10.9.0.5 Port: 52744
Destination: 10.9.0.6 Port: 23
   Protocol: TCP
Payload:
                        1

Source: 10.9.0.5 Port: 52744
Destination: 10.9.0.6 Port: 23
   Protocol: TCP
Payload:
                        2

Source: 10.9.0.5 Port: 52744
Destination: 10.9.0.6 Port: 23
   Protocol: TCP
Payload:
                        3

Source: 10.9.0.5 Port: 52744
Destination: 10.9.0.6 Port: 23
   Protocol: TCP
Payload:
                        4
```

This sniffer program was modified to show the payload (raw data) of the packets travelling between 2 hosts. This allowed us to capture the password when using telnet.

Task 2.2A

spoof.c:

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <unistd.h>

// checksum for IP/ICMP headers
unsigned short checksum(void *b, int len) {
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    for (sum = 0; len > 1; len -= 2)
        sum += *buf++;
    if (len == 1)
        sum += *(unsigned char *)buf;
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;
    return result;
}

int main() {
    int sock;
    struct sockaddr_in target_addr;
    char packet[1024];

    // make raw socket
    sock = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sock < 0) {
        perror("Socket creation failed");
```

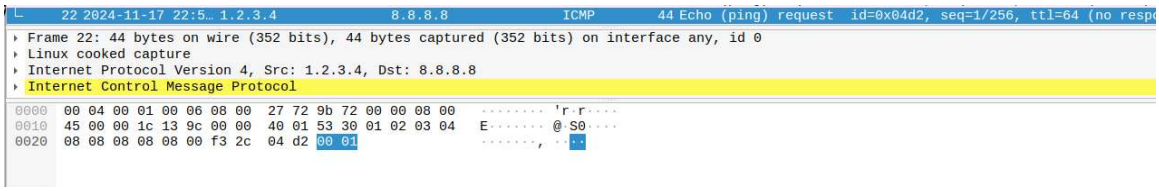
```
        exit(1);
    }

    int opt = 1;
    if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &opt, sizeof(opt)) < 0) {
        perror("setsockopt failed");
        exit(1);
    }

    memset(packet, 0, sizeof(packet));

    // IP Header
    struct iphdr *ip_header = (struct iphdr *)packet;
    struct icmphdr *icmp_header = (struct icmphdr *)(packet + sizeof(struct iphdr));


    ip_header->version = 4;
    ip_header->ihl = 5;
    ip_header->tos = 0;
    ip_header->tot_len = htons(sizeof(struct iphdr) + sizeof(struct icmphdr));
    ip_header->id = htonl(54321); // random
    ip_header->frag_off = 0;
    ip_header->ttl = 64;
    ip_header->protocol = IPPROTO_ICMP;
    ip_header->saddr = inet_addr("1.2.3.4"); // spoofed source IP
```

```c
    ip_header->saddr = inet_addr("1.2.3.4"); // spoofed source IP
    ip_header->daddr = inet_addr("8.8.8.8"); // target

    // icmp Header
    icmp_header->type = ICMP_ECHO;
    icmp_header->code = 0;
    icmp_header->un.echo.id = htons(1234); // fake ID
    icmp_header->un.echo.sequence = htons(1);
    icmp_header->checksum = 0;
    icmp_header->checksum = checksum(icmp_header, sizeof(struct icmphdr));

    //point to 8.8.8.8 address
    target_addr.sin_family = AF_INET;
    target_addr.sin_addr.s_addr = inet_addr("8.8.8.8");

        //send
    if (sendto(sock, packet, sizeof(struct iphdr) + sizeof(struct icmphdr), 0,
               (struct sockaddr *)&target_addr, sizeof(target_addr)) < 0) {
        perror("Packet send failed");
    } else {
        printf("Spoofed packet sent successfully!\n");
    }

    close(sock);
    return 0;
}
```

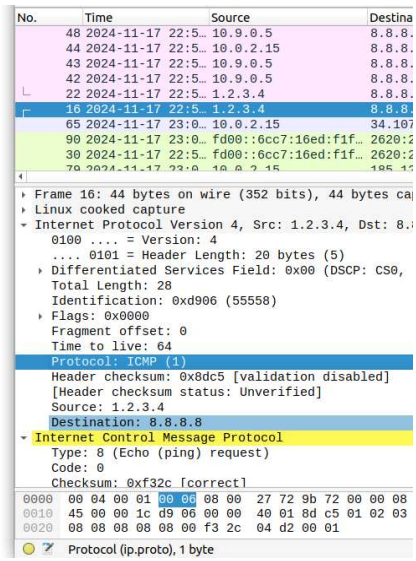
- This program will create a raw socket to send to the spoofed ICMP echo request packet. It manually builds the IP and ICMP headers, calculates checksum and then sends the packet to the destination IP. This makes it so it looks like the packet is coming from a different IP address. Lets test this and prove it works:

```
[11/17/24]seed@VM:~/.../Labsetup$ touch spoof.c
[11/17/24]seed@VM:~/.../Labsetup$ gcc -o spoof spoof.c
[11/17/24]seed@VM:~/.../Labsetup$ sudo ./spoof
Spoofed packet sent successfully!
```

Wireshark trace:




```c
51     ip_header->version = 4;
52     ip_header->ihl = 5;
53     ip_header->tos = 0;
54     ip_header->tot_len = htons(sizeof(struct iphdr) +
  sizeof(struct icmphdr));
55     ip_header->id = htonl(54321); // random
56     ip_header->frag_off = 0;
57     ip_header->ttl = 64;
58     ip_header->protocol = IPPROTO_ICMP;
59     ip_header->saddr = inet_addr("1.2.3.4"); // spoofed source IP
60     ip_header->daddr = inet_addr("8.8.8.8"); // target
61
62     // icmp Header
63     icmp_header->type = ICMP_ECHO;
64     icmp_header->code = 0;
65     icmp_header->un.echo.id = htons(1234); // fake ID
66     icmp_header->un.echo.sequence = htons(1);
67     icmp_header->checksum = 0;
68     icmp_header->checksum = checksum(icmp_header, sizeof(struct
  icmphdr));
69
70     //point to 8.8.8.8 address
71     target_addr.sin_family = AF_INET;
72     target_addr.sin_addr.s_addr = inet_addr("8.8.8.8");
73
```



- It seems that the fake details we specified in our code was sent to the destination IP which indicates that the spoof worked.

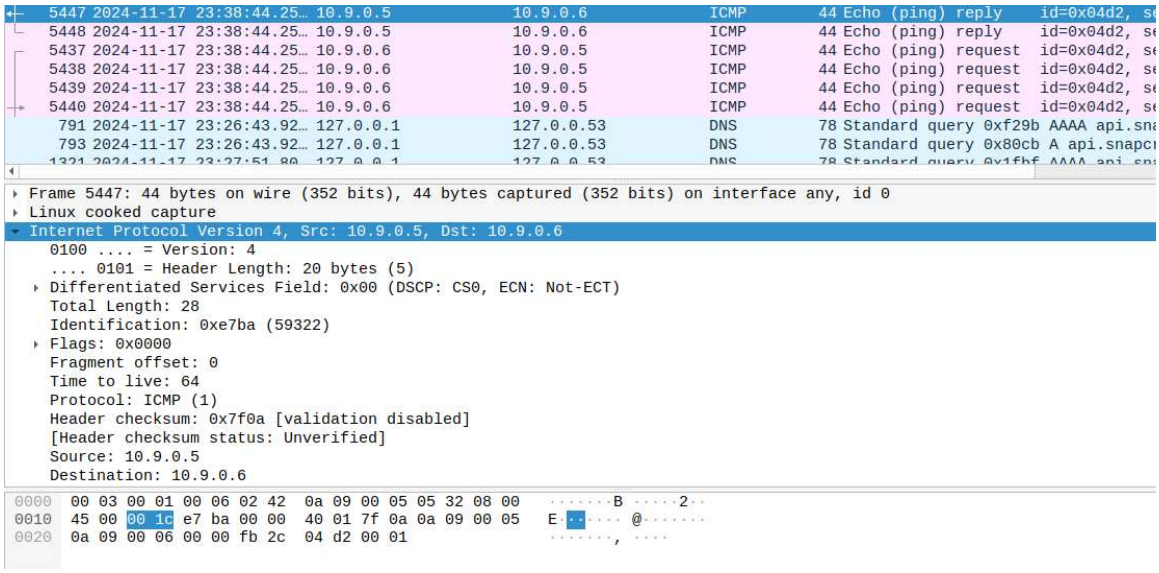
Task 2.2B

Edit to spoof.c:

```
    ip_header->version = 4;
    ip_header->ihl = 5;
    ip_header->tos = 0;
    ip_header->tot_len = htons(sizeof(struct iphdr) + sizeof(struct icmphdr));
    ip_header->id = htonl(54321); // random
    ip_header->frag_off = 0;
    ip_header->ttl = 64;
    ip_header->protocol = IPPROTO_ICMP;
    ip_header->saddr = inet_addr("10.9.0.6"); // spoofed so IPS
    ip_header->daddr = inet_addr("10.9.0.5"); // target

    // icmp Header
    icmp_header->type = ICMP_ECHO;
    icmp_header->code = 0;
    icmp_header->un.echo.id = htons(1234); // fake ID
    icmp_header->un.echo.sequence = htons(1);
    icmp_header->checksum = 0;
    icmp_header->checksum = checksum(icmp_header, sizeof(st icmphdr));

    //point to 10.9.0.5 address
    target_addr.sin_family = AF_INET;
    target_addr.sin_addr.s_addr = inet_addr("10.9.0.5");

        //send
    if (sendto(sock, packet, sizeof(struct iphdr) + sizeof( icmphdr), 0,
```

- We are just changing the destination and source IP with that of our containers hostA and hostB

Wireshark:



- Seems that we have succefully spoofed Host B

Question 4. Can you set the IP packet length field to an arbitrary value, regardless of how big the
actual packet is?

- Yes, the IP packet length field can be set to any arbitrary value. However, when the packet is sent, the total packet length will be overwritten with its actual size.

Question 5. Using the raw socket programming, do you have to calculate the checksum for the IP
header?

- With raw sockets, the kernel can handle the IP header checksum calculation by default. This is the case unless you explicitly change it. If you choose to calculate it manually, you'll need to implement a checksum method.

Question 6. Why do you need the root privilege to run the programs that use raw sockets? Where
does the program fail if executed without the root privilege?

- Root privileges are required because raw sockets allow direct modification of protocol header fields, which regular users don't have permission to do. Root users can set any header field and access the network interface, including putting it into promiscuous mode. Without root privileges, the program will fail at the

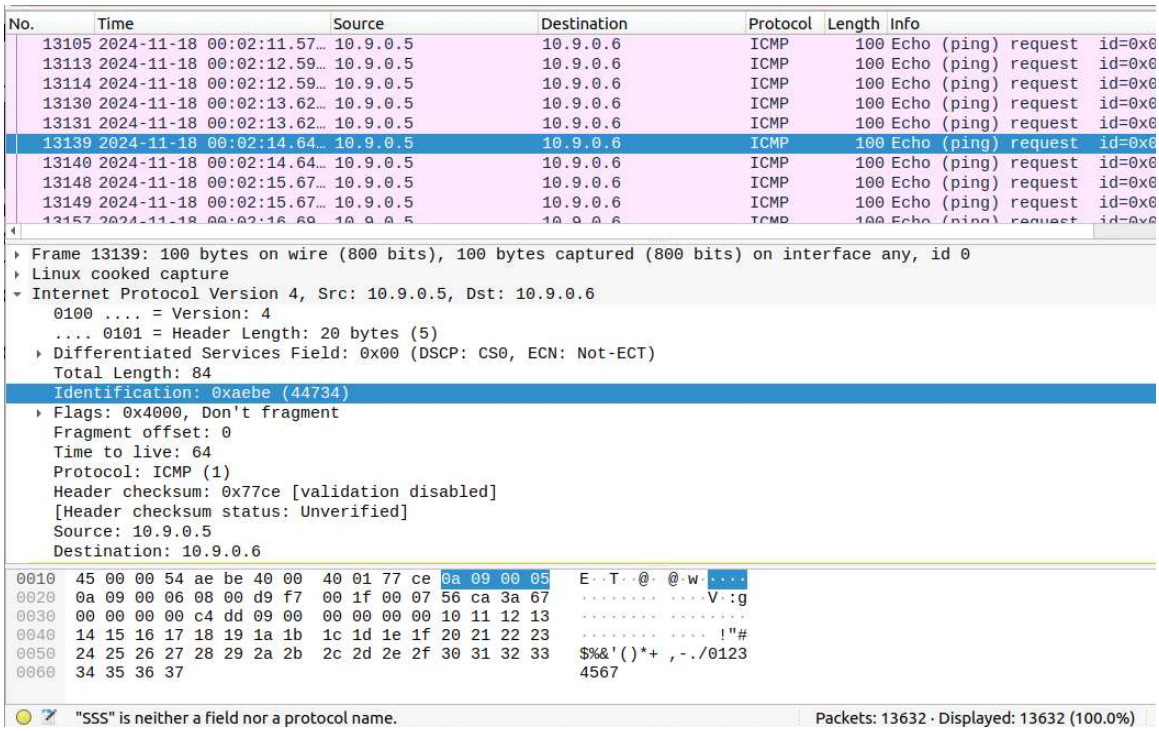
socket creation step.

Task 2.3

- We will run sniffspoof.c then ping host B through host A :

```
root@6c35e2fab862:/# ping 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=0.079 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.045 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.051 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.046 ms
64 bytes from 10.9.0.6: icmp_seq=5 ttl=64 time=0.051 ms
64 bytes from 10.9.0.6: icmp_seq=6 ttl=64 time=0.041 ms
64 bytes from 10.9.0.6: icmp_seq=7 ttl=64 time=0.053 ms
64 bytes from 10.9.0.6: icmp_seq=8 ttl=64 time=0.058 ms
64 bytes from 10.9.0.6: icmp_seq=9 ttl=64 time=0.054 ms
64 bytes from 10.9.0.6: icmp_seq=10 ttl=64 time=0.048 ms
64 bytes from 10.9.0.6: icmp_seq=11 ttl=64 time=0.053 ms
64 bytes from 10.9.0.6: icmp_seq=12 ttl=64 time=0.047 ms
64 bytes from 10.9.0.6: icmp_seq=13 ttl=64 time=0.050 ms
64 bytes from 10.9.0.6: icmp_seq=14 ttl=64 time=0.046 ms
64 bytes from 10.9.0.6: icmp_seq=15 ttl=64 time=0.048 ms
64 bytes from 10.9.0.6: icmp_seq=16 ttl=64 time=0.052 ms
64 bytes from 10.9.0.6: icmp_seq=17 ttl=64 time=0.472 ms
```

- **Wireshark:**



Explanation of the following code is provided in comments:

sniffspoof.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pcap.h>
#include <netinet/ip.h>
#include <netinet/in.h>
#include <netinet/ip_icmp.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

#define IP_HDR_LEN 20
#define ICMP_HDR_LEN 8
#define BUFSIZE 1024

// function to callculate the checksum of the packet
unsigned short checksum(void *b, int len) {
```

```
    unsigned short *buf = b;
    unsigned int sum = 0;
    unsigned short result;

    // sum all 16-bit words
    for (sum = 0; len > 1; len -= 2) {
        sum += *buf++;
    }

    // if theres an odd byte left then add it to the sum
    if (len == 1) {
        sum += *(unsigned char *)buf;
    }

    // perform end-around carry and complement
    sum = (sum >> 16) + (sum & 0xFFFF);
    sum += (sum >> 16);
    result = ~sum;  // invert the sum to get the checksum
    return result;
}

// function to send a spoofed ICMP echo reply
void send_icmp_reply(char *target_ip, char *src_ip, char *data, int data_len) {
    int sock;
    struct sockaddr_in dest;
    char packet[BUFSIZE];
    struct ip *ip_header = (struct ip *)packet;
    struct icmp *icmp_header = (struct icmp *)(packet + IP_HDR_LEN);

    // ip header
    ip_header->ip_hl = 5;  // IP header length (in 32-bit words)
    ip_header->ip_v = 4;   // IP version (IPv4)
    ip_header->ip_tos = 0; // type of service
    ip_header->ip_len = IP_HDR_LEN + ICMP_HDR_LEN + data_len;  // Total packet
size
    ip_header->ip_id = htonl(54321);  // Arbitrary ID for fragmentation
    ip_header->ip_off = 0;   // no fragmentation
    ip_header->ip_ttl = 64; // time to live (TTL)
    ip_header->ip_p = IPPROTO_ICMP;  // protocol (ICMP)
    ip_header->ip_sum = 0;  // checksum (calculated later)
```

```
    ip_header->ip_src.s_addr = inet_addr(src_ip); // Source IP address
    ip_header->ip_dst.s_addr = inet_addr(target_ip); // Destination IP address

    // callculate the IP checksum
    ip_header->ip_sum = checksum(ip_header, IP_HDR_LEN);

    // prepare the ICMP header
    icmp_header->icmp_type = ICMP_ECHOREPLY;  // ICMP Echo Reply type
    icmp_header->icmp_code = 0;  // No code
    icmp_header->icmp_id = 1234;  // arbitrary ID
    icmp_header->icmp_seq = 1;  // ssequence number
    icmp_header->icmp_cksum = 0;  // checksum (calculated later)

    //calculate the ICMP checksum
    icmp_header->icmp_cksum = checksum(icmp_header, ICMP_HDR_LEN + data_len);

    //  create a raw socket for sending the packet
    sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
    if (sock < 0) {
        perror("Socket Error");
        exit(1);
    }

    //  setup the destination address for the reply
    dest.sin_family = AF_INET;
    dest.sin_port = 0;
    dest.sin_addr.s_addr = inet_addr(target_ip); // Target's IP address

    //copy the IP and ICMP headers into the packet buffer
    memcpy(packet, ip_header, IP_HDR_LEN);
    memcpy(packet + IP_HDR_LEN, icmp_header, ICMP_HDR_LEN);
    memcpy(packet + IP_HDR_LEN + ICMP_HDR_LEN, data, data_len);  // Add the
ICMP data

    //send the crafted ICMP echo reply packet
    if (sendto(sock, packet, ip_header->ip_len, 0, (struct sockaddr *)&dest, sizeof(dest))
< 0) {
        perror("Sendto failed");
    }
```

```
    close(sock);  // close the socket
}

// packet sniffing callback function
void packet_handler(unsigned char *user_data, const struct pcap_pkthdr *pkthdr, const
unsigned char *packet) {
    struct ip *ip_header = (struct ip *)(packet + 14);  // skip Ethernet header (14 bytes)
    struct icmp *icmp_header = (struct icmp *)(packet + 14 + IP_HDR_LEN);  // skip IP
header

    // check if the packet is an ICMP Echo Request
    if (icmp_header->icmp_type == ICMP_ECHO) {
        char target_ip[INET_ADDRSTRLEN];
        char src_ip[INET_ADDRSTRLEN];

        // convert IP addresses to strings
        inet_ntop(AF_INET, &(ip_header->ip_src), src_ip, INET_ADDRSTRLEN);  //
Source IP
        inet_ntop(AF_INET, &(ip_header->ip_dst), target_ip, INET_ADDRSTRLEN);  //
Target IP

        printf("Captured ICMP Echo Request from %s to %s\n", src_ip, target_ip);

        // send a spoofed ICMP Echo Reply back to the source IP
        send_icmp_reply(src_ip, target_ip, (unsigned char *)icmp_header +
ICMP_HDR_LEN, pkthdr->len - 14 - IP_HDR_LEN);
    }
}

int main() {
    char *dev = "enp0s3";  // etwork interface
    char errbuf[PCAP_ERRBUF_SIZE];
    pcap_t *handle;

    // open the device for sniffing packets
    handle = pcap_open_live(dev, BUFSIZE, 1, 1000, errbuf);
    if (handle == NULL) {
        fprintf(stderr, "Error opening device %s: %s\n", dev, errbuf);
        return 1;
    }
```

```
    // start sniffing and handle each packet with the packet_handler function
    printf("Starting sniffing for ICMP Echo Requests...\n");
    pcap_loop(handle, 0, packet_handler, NULL);

    // close the pcap handle after sniffing
    pcap_close(handle);

    return 0;
}
```